```
# Step 1: Import Necessary Libraries
import pandas as pd
import numpy as np

# Step 2: Create Dataset Manually (Banknote-like dataset)
data = pd.DataFrame({
    'variance': [2.3, 1.2, 3.1, 2.0, 1.5, 3.6, 2.9, 0.5, 1.1, 3.0],
    'skewness': [6.2, 5.1, 7.4, 6.0, 5.5, 7.8, 6.9, 3.2, 4.8, 7.0],
    'curtosis': [1.5, 1.1, 2.0, 1.8, 1.2, 2.3, 2.1, 0.8, 1.0, 2.2],
    'entropy': [0.2, 0.1, 0.3, 0.25, 0.15, 0.35, 0.28, 0.05, 0.12, 0.30],
    'Class': [0, 0, 1, 1, 0, 1, 1, 0, 0, 1]
})

print("First 10 rows:\n", data.head(10))

# Step 3: Display Dataset Dimensions
print("\nShape:", data.shape)

# Step 4: Display Descriptive Statistics
print("\nDescription:\n", data.describe())

# Step 5: Import Train-Test Split
from sklearn.model_selection import train_test_split

# Step 6: Prepare Data
X = data.drop('Class', axis=1)
y = data['Class']

# ==========================================================
# FUNCTION TO RUN MODEL WITH DIFFERENT ACTIVATIONS
# ==========================================================
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, classification_report

def run_model(x_train, x_test, y_train, y_test, activation_name):
    print(f"\n--- Activation: {activation_name} ---")

    mlp = MLPClassifier(max_iter=500, activation=activation_name, random_state=0)
    mlp.fit(x_train, y_train)

    pred = mlp.predict(x_test)

    print("Predictions:", pred)

    cm = confusion_matrix(y_test, pred)
    print("Confusion Matrix:\n", cm)

    print("Classification Report:\n", classification_report(y_test, pred))


# ==========================================================
# Step 7–14: TRAIN TEST SPLIT = 80-20
# ==========================================================
print("\n========== TRAIN-TEST SPLIT 80-20 ==========")

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

# Run for all activation functions
for act in ['relu', 'logistic', 'tanh', 'identity']:
    run_model(x_train, x_test, y_train, y_test, act)


# ==========================================================
# Step 15: TRAIN TEST SPLIT = 70-30
# ==========================================================
print("\n========== TRAIN-TEST SPLIT 70-30 ==========")

x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)
```

```
# Run again for all activation functions
for act in ['relu', 'logistic', 'tanh', 'identity']:
    run_model(x_train, x_test, y_train, y_test, act)
```

```
First 10 rows:
    variance  skewness  curtosis  entropy  Class
0       2.3       6.2       1.5     0.20      0
1       1.2       5.1       1.1     0.10      0
2       3.1       7.4       2.0     0.30      1
3       2.0       6.0       1.8     0.25      1
4       1.5       5.5       1.2     0.15      0
5       3.6       7.8       2.3     0.35      1
6       2.9       6.9       2.1     0.28      1
7       0.5       3.2       0.8     0.05      0
8       1.1       4.8       1.0     0.12      0
9       3.0       7.0       2.2     0.30      1

Shape: (10, 5)

Description:
         variance   skewness   curtosis    entropy      Class
count  10.000000  10.000000  10.000000  10.000000  10.000000
mean    2.120000   5.990000   1.600000   0.210000   0.500000
std     1.026104   1.390004   0.549747   0.100995   0.527046
min     0.500000   3.200000   0.800000   0.050000   0.000000
25%     1.275000   5.200000   1.125000   0.127500   0.000000
50%     2.150000   6.100000   1.650000   0.225000   0.500000
75%     2.975000   6.975000   2.075000   0.295000   1.000000
max     3.600000   7.800000   2.300000   0.350000   1.000000

========== TRAIN-TEST SPLIT 80-20 ==========

--- Activation: relu ---
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:
  warnings.warn(
Predictions: [1 0]
Confusion Matrix:
 [[1 0]
 [0 1]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2


--- Activation: logistic ---
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning:
  warnings.warn(
Predictions: [1 0]
Confusion Matrix:
 [[1 0]
 [0 1]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
```